`cw` — parameter study

Generated by `sweep/analyze_sweep.py`

2026-08-02

Setup

346 runs of `./cw`, 1,000 instances each, generated with the Kool/NeuOpt law (coordinates $U[0,1]^2$, demands $U\{1,\dots,9\}$, capacity from `default_capacity`). Unless a study says otherwise, $n = 100$ and `--sa-steps` = 100,000.

Every comparison is paired. Instance k is generated from `seed` + k (`cw.c:2064`), so all runs of a study see byte-identical instances. Rather than comparing two means with their independent standard deviations, we compare $\text{cost}_i(A)$ with $\text{cost}_i(B)$ on the same instance i and report the mean of the 1,000 differences with a 95 % confidence interval, plus a distribution-free sign test. This matters: the between-instance spread ($\sigma \approx 1.9$ at $n = 100$) is more than an order of magnitude larger than the effects being measured.

In every table, Δ is that paired mean as a percentage of the baseline; **bold** means the CI excludes zero *and* the sign test rejects at 5 %; `win/loss` counts instances strictly improved and strictly worsened. **Negative is better** — these are costs.

Contents

1 Overview: what each knob is worth

Each row below varies one knob with everything else left at its default, and reports the best setting the sweep found for it. Negative is better. The sections that follow explain what each knob does and give the full grids.

> **Read the confidence intervals, not the point estimates.** A setting is listed *because* it came out lowest, so its Δ is optimistically biased — the more values a row sweeps, the stronger the bias. A row whose CI straddles zero is a knob that did nothing here. This is why Section 12 re-runs the winners on a seed the sweep never saw.

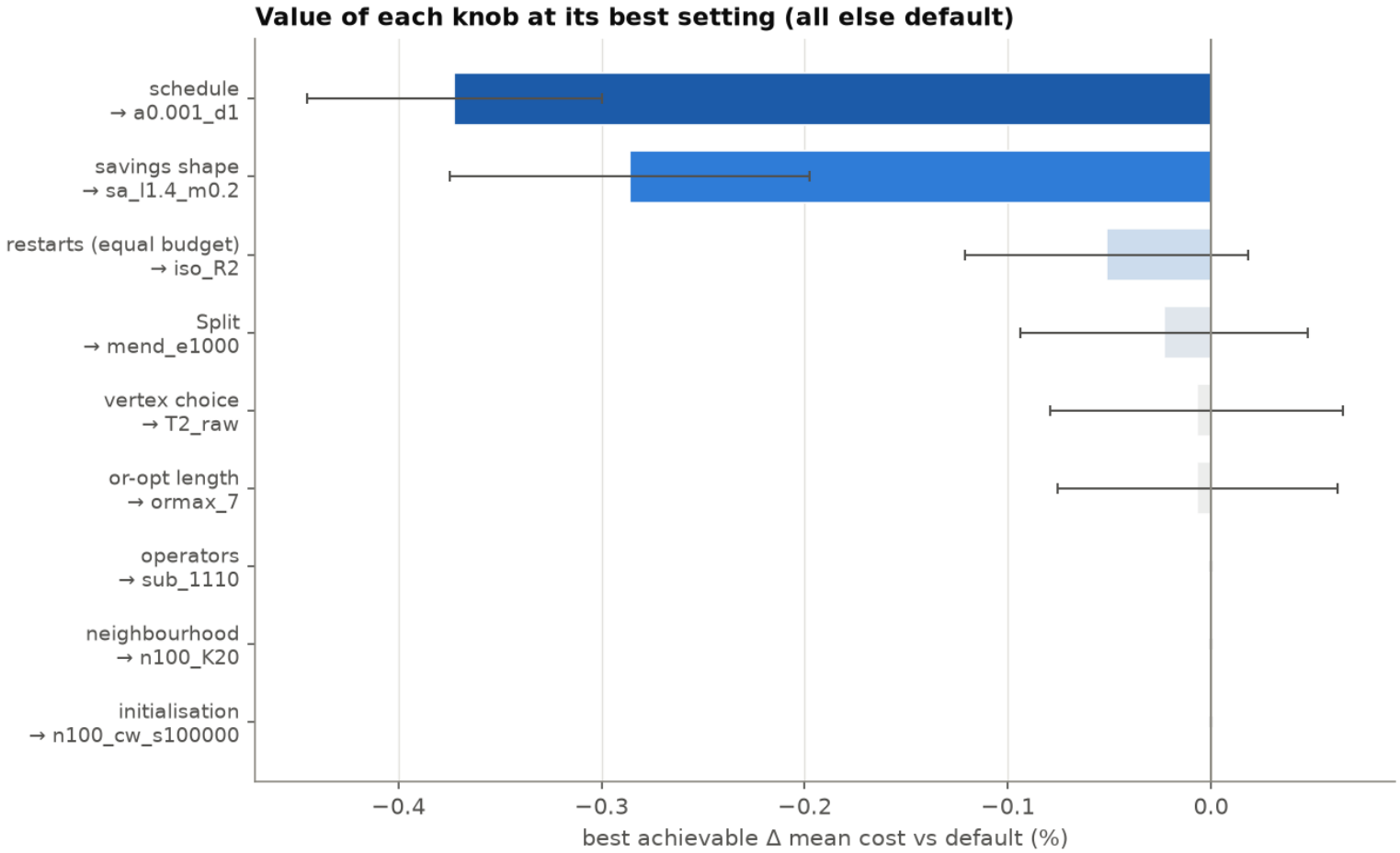

Figure 1: Best achievable improvement per knob, with 95 % CIs. Bars that cross zero are knobs with no measurable effect on this instance family at this budget.

knob	best setting found	Δ (%)	95 % CI	win/loss
schedule	`a0.001_d1`	**-0.372**	[-0.445, -0.300]	611/382
savings shape	`sa_l1.4_m0.2`	**-0.286**	[-0.375, -0.197]	565/435
restarts (equal budget)	`iso_R2`	-0.051	[-0.121, +0.018]	508/491
Split	`mend_e1000`	-0.023	[-0.094, +0.048]	501/495
vertex choice	`T2_raw`	-0.007	[-0.079, +0.065]	490/506
or-opt length	`ormax_7`	-0.007	[-0.075, +0.062]	503/496
operators	`sub_1110`	+0.000	[+0.000, +0.000]	0/0
neighbourhood	`n100_K20`	+0.000	[+0.000, +0.000]	0/0
initialisation	`n100_cw_s100000`	+0.000	[+0.000, +0.000]	0/0

Table 1: Each knob at its best setting, paired against its own default.

2 Initialisation: where the annealing starts from

What it controls. `--init` chooses the solution the simulated annealing starts from. `cw` (the default) runs the Clarke & Wright savings construction: every pair of customers gets a score $s(i,j) = d_{0i} + d_{0j} - \lambda\, d_{ij} + \mu\, |d_{0i} - d_{0j}|$, measuring how much is saved by serving i and j on one route instead of two, and merges route endpoints greedily in decreasing score while capacity allows. `random` throws that away: it shuffles the customers and cuts the permutation into routes at the capacity limit (first fit). Under `--restarts` each restart draws its own permutation.

The question. A random start is 100–380 % worse before annealing. Does the annealing erase that, and how much budget does it cost to do so?

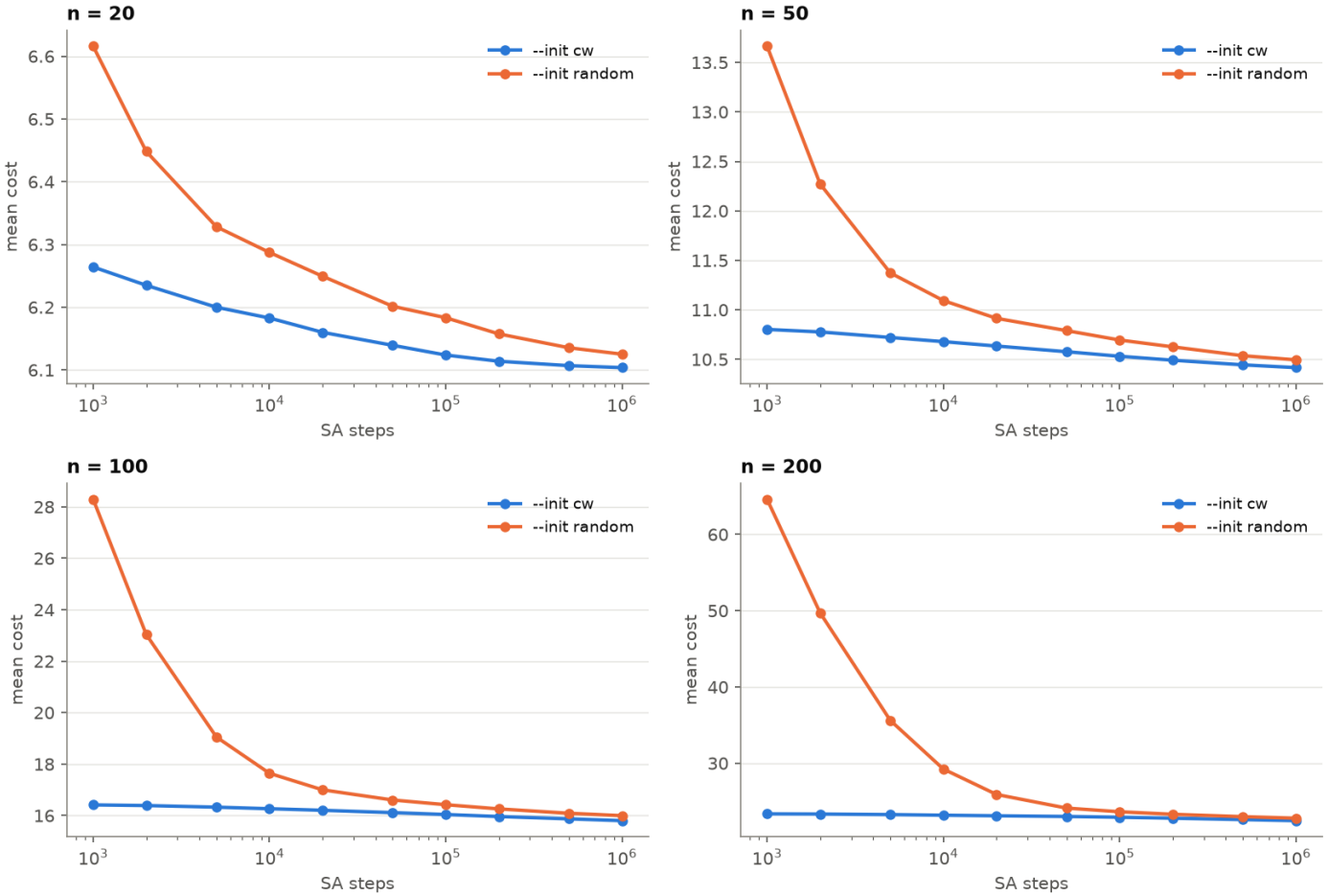

Figure 2: Mean cost against the annealing budget, for both initialisations, at four dimensions. Same budget on both sides of each panel.

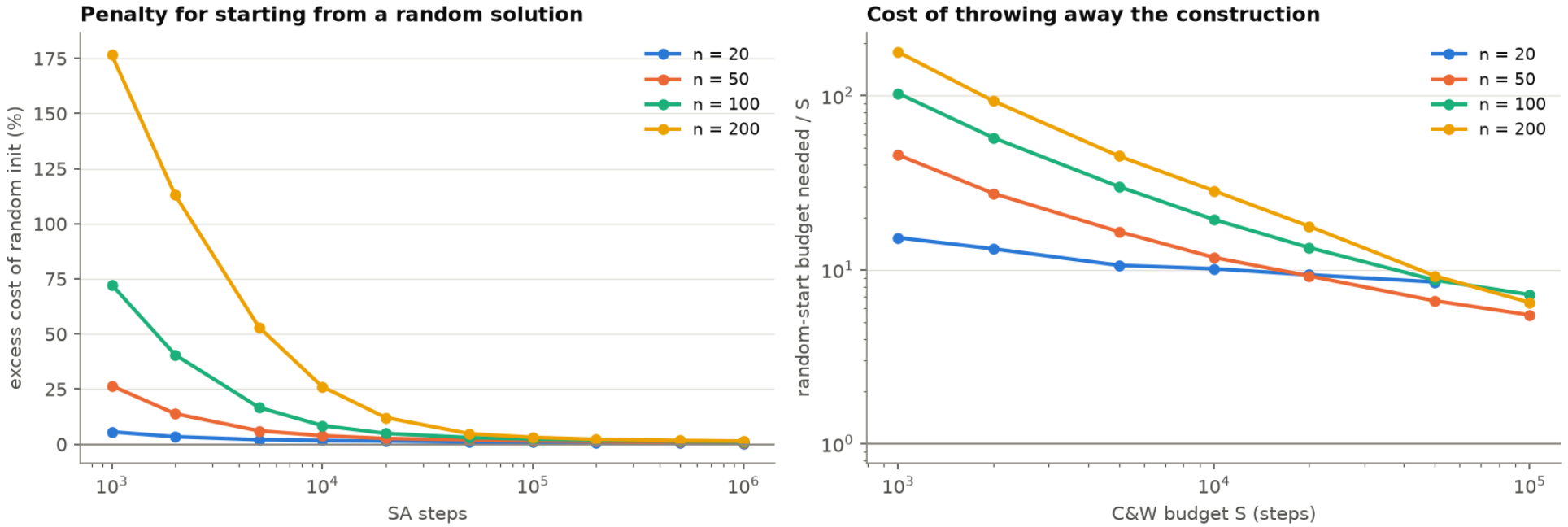

Figure 3: Left: paired excess of the random start over Clarke & Wright at equal budget, with a 95 % band. Right: how many steps the random start needs to reach what Clarke & Wright reaches at S steps.

n	**excess at 10^3 (%)**	**excess at 10^6 (%)**	**steps to catch up**
20	+5.62	+0.35	not within 10^6
50	+26.48	+0.75	not within 10^6
100	+72.36	+1.23	not within 10^6
200	+176.60	+1.46	not within 10^6

Table 2: Penalty for a random start, paired against the C&W start at the same budget. ‘Catch up’ is the first budget at which the upper end of the 95 % CI reaches zero.

n	**at smallest S**	**mid-range S**	**at largest S**
20	15×	10×	9×
50	46×	12×	5×
100	103×	19×	7×
200	178×	28×	6×

Table 3: Budget multiplier: steps the random start needs to match the C&W start, relative to the C&W budget. A ratio of 1 means the head start has been fully repaid.

Reading. The random start converges to parity but never overtakes: it buys no extra diversity that the annealing can exploit. The gap closes late and costs dearly along the way — at $n = 200$ it still needs several times the budget at 10^5 steps. Keep the construction.

3 Move operators

What it controls. `--ops r,s,t,o` sets the relative probabilities of the four neighbourhood moves the annealing draws from. Each move first picks a vertex u (Section 8) and a partner v near it (Section 4), then:

- **relocate** (`mv_relocate`, `cw.c:980`) — take u out of its route and re-insert it next to v, possibly in another route.
- **swap** (`mv_swap`, `cw.c:1017`) — exchange the positions of u and v.
- **2-opt** (`mv_2opt`, `cw.c:1137`) — replace the two edges carried by u and v by the other pairing; inside one route this reverses a segment, across two routes it exchanges their tails.
- **or-opt** (`mv_oropt`, `cw.c:1082`) — move a whole run of 2 to `--or-max` consecutive customers starting at u, optionally reversed, next to v.

Every move is accepted by the Metropolis rule on its exact cost delta. The default is `1,1,1,0`: or-opt is present in the code but switched off.

The question. Is the default subset the right one, is the uniform weighting right, and is or-opt off for a good reason?

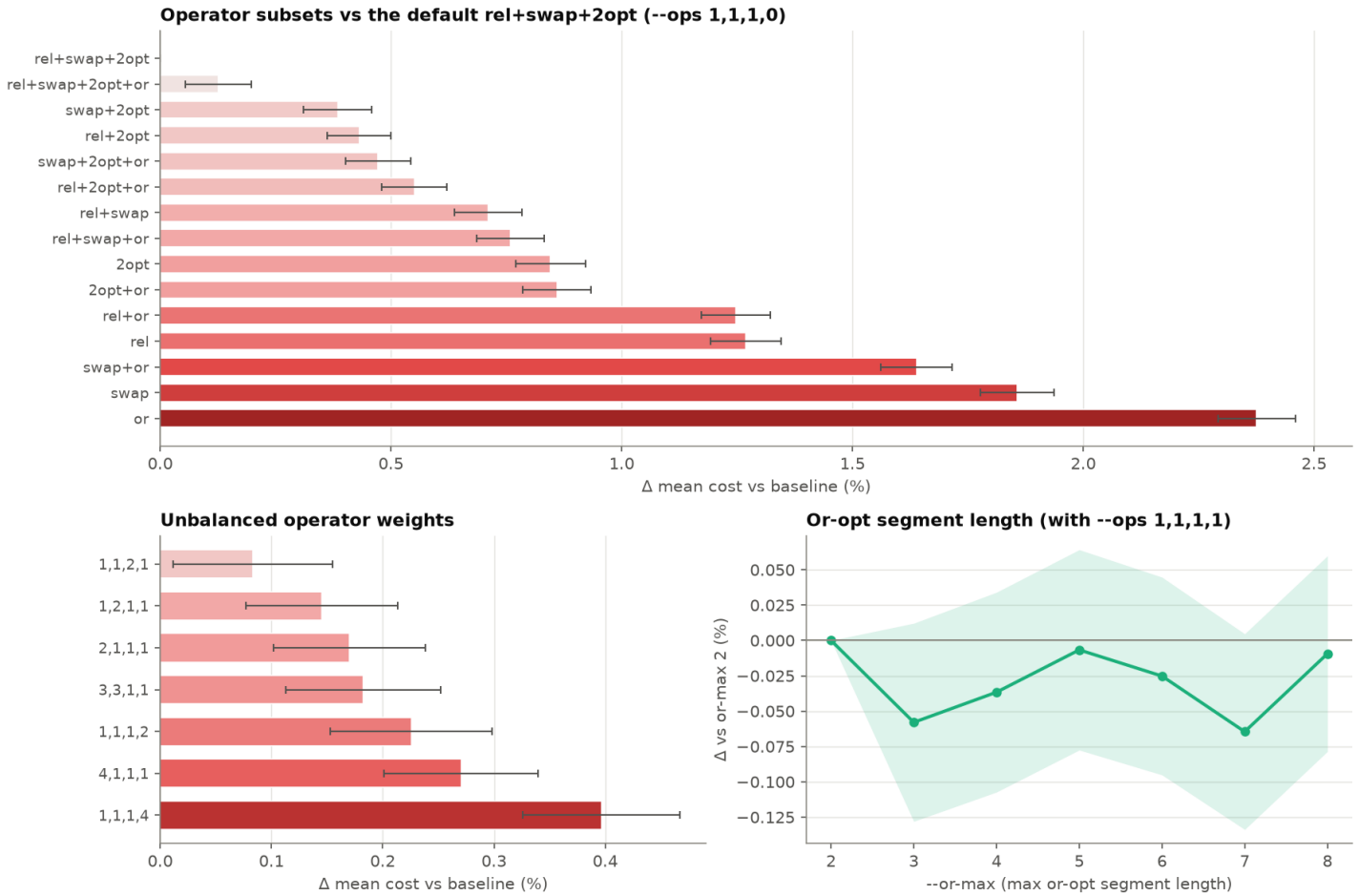

Figure 4: Top: all 15 non-empty operator subsets. Bottom left: unbalanced weightings. Bottom right: the or-opt segment length cap. Bars are paired deltas against the default with 95 % CIs.

subset	mean cost	Δ (%)	95 % CI	win/loss
rel+swap+2opt	16.03992	+0.000	[+0.000, +0.000]	0/0
rel+swap+2opt+or	16.06019	**+0.126**	[+0.055, +0.198]	456/541
swap+2opt	16.10158	**+0.384**	[+0.310, +0.459]	346/650
rel+2opt	16.10911	**+0.431**	[+0.362, +0.500]	334/663
swap+2opt+or	16.11568	**+0.472**	[+0.402, +0.542]	330/668
rel+2opt+or	16.12824	**+0.551**	[+0.480, +0.621]	305/692
rel+swap	16.15384	**+0.710**	[+0.637, +0.783]	252/743
rel+swap+or	16.16167	**+0.759**	[+0.686, +0.832]	241/753
2opt	16.17559	**+0.846**	[+0.770, +0.921]	234/763
2opt+or	16.17781	**+0.860**	[+0.786, +0.933]	202/795
rel+or	16.23987	**+1.247**	[+1.172, +1.321]	126/870
rel	16.24344	**+1.269**	[+1.192, +1.346]	137/859
swap+or	16.30282	**+1.639**	[+1.562, +1.716]	63/929
swap	16.33785	**+1.857**	[+1.778, +1.937]	34/957
or	16.42100	**+2.376**	[+2.292, +2.460]	7/981

Table 4: All 15 non-empty operator subsets, paired against the default `--ops 1,1,1,0`. Bold = significant at 95 % and by sign test.

weights	mean cost	Δ (%)	95 % CI	win/loss
`1,1,2,1`	16.05328	**+0.083**	[+0.012, +0.155]	454/543
`1,2,1,1`	16.06320	**+0.145**	[+0.077, +0.214]	436/556
`2,1,1,1`	16.06718	**+0.170**	[+0.102, +0.238]	430/563
`3,3,1,1`	16.06921	**+0.183**	[+0.113, +0.252]	418/579
`1,1,1,2`	16.07610	**+0.226**	[+0.153, +0.298]	419/579
`4,1,1,1`	16.08330	**+0.270**	[+0.201, +0.340]	403/593
`1,1,1,4`	16.10354	**+0.397**	[+0.326, +0.467]	355/640

Table 5: Unbalanced weightings, against the same default.

`--or-max`	**mean cost**	**Δ (%)**	**95 % CI**	**win/loss**
2	16.06950	+0.000	[+0.000, +0.000]	0/0
3	16.06019	-0.058	[-0.128, +0.012]	527/472
4	16.06362	-0.037	[-0.107, +0.034]	530/467
5	16.06843	-0.007	[-0.078, +0.064]	500/496
6	16.06545	-0.025	[-0.095, +0.045]	485/512
7	16.05913	-0.065	[-0.134, +0.005]	529/465
8	16.06799	-0.009	[-0.079, +0.060]	497/497

Table 6: Or-opt segment length, with or-opt enabled (`--ops 1,1,1,1`). Valid range is 2–8 (`cw.c:2042`).

Reading. The default subset wins: every other subset is worse, and turning or-opt on costs a small but significant amount. Or-opt overlaps with relocate (a length-1 relocate is the same move) while being more expensive per draw, so at a fixed step count it buys less. `--or-max` is then irrelevant — its whole range sits inside the noise. Leaving or-opt off is the right default.

4 Candidate neighbourhood

What it controls. Once a move has picked the vertex u, it needs a partner v. `sa_cand` (`cw.c:971`) draws v uniformly from the K nearest neighbours of u, with $K =$ `--sa-knn`; at $K = 0$ it draws uniformly from all customers. This is what keeps a move geometrically local, so that the cost delta has a chance of being negative. K is clamped to $n - 1$ (`cw.c:1399`).

> K does double duty: at $K = 0$ the kNN lists are not built, and `cw.c:1400` then *silently* forces uniform vertex selection as well, whatever `--pick` says. So $K = 0$ disables two mechanisms, not one — see Section 8.

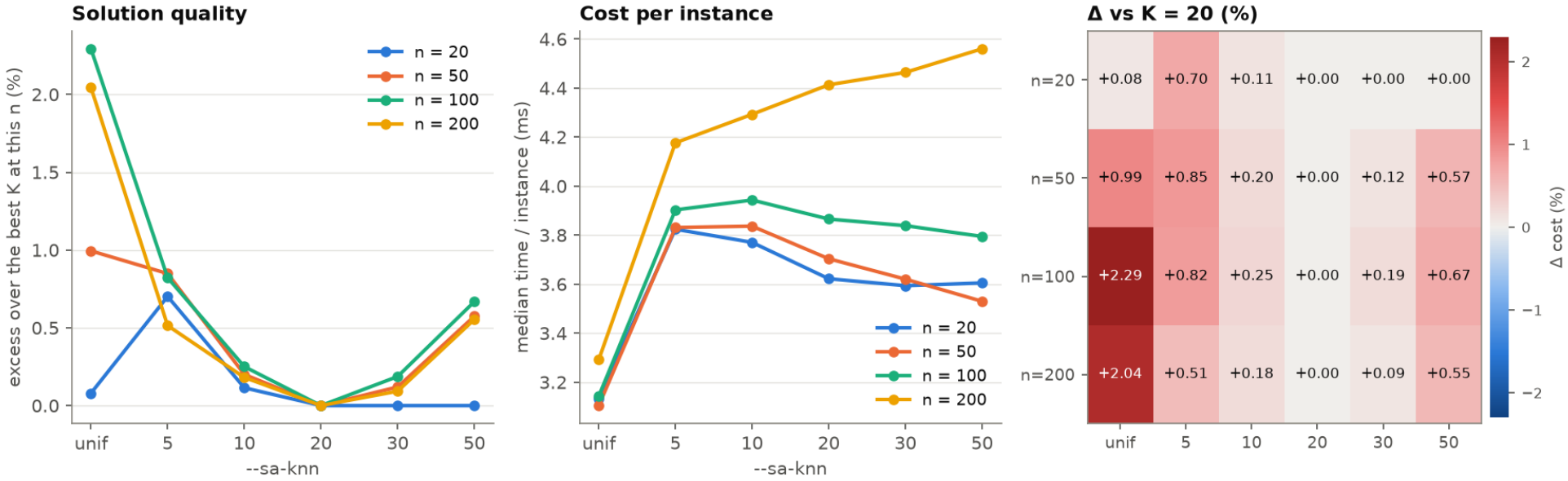

Figure 5: Quality (left, as excess over the best K at that n), cost per instance (middle) and the paired delta against $K = 20$ (right).

Table 7: `--sa-knn` across dimension, paired against $K = 20$. The effective K is $\min(K, n-1)$, which is why the rows for $n = 20$ at $K \geq 20$ are exactly identical runs.

n	K	effective K	mean cost	Δ (%)	95 % CI	win/loss	ms/inst	
20	0	uniform	6.12870	**+0.080**	[+0.015, +0.145]	176/233	3.133	
20	5	5	6.16679	**+0.702**	[+0.593, +0.811]	121/362	3.824	
20	10	10	6.13083	+0.114	[+0.045, +0.184]	186/191	3.770	
20	20	19	6.12382	+0.000	[+0.000, +0.000]	0/0	3.622	⋆
20	30	19	6.12382	+0.000	[+0.000, +0.000]	0/0	3.594	
20	50	19	6.12382	+0.000	[+0.000, +0.000]	0/0	3.605	
50	0	uniform	10.63593	**+0.994**	[+0.888, +1.101]	236/722	3.104	
50	5	5	10.62058	**+0.848**	[+0.753, +0.944]	255/705	3.831	
50	10	10	10.55234	**+0.200**	[+0.112, +0.289]	422/516	3.836	
50	20	20	10.53122	+0.000	[+0.000, +0.000]	0/0	3.704	⋆
50	30	30	10.54381	**+0.120**	[+0.024, +0.215]	432/528	3.620	
50	50	49	10.59167	**+0.574**	[+0.470, +0.678]	326/627	3.530	
100	0	uniform	16.40774	**+2.293**	[+2.211, +2.376]	11/973	3.143	

n	K	effective K	mean cost	Δ (%)	95 % CI	win/loss	ms/inst	
100	5	5	16.17175	**+0.822**	[+0.751, +0.893]	226/772	3.903	
100	10	10	16.08009	**+0.250**	[+0.183, +0.318]	402/594	3.943	
100	20	20	16.03992	+0.000	[+0.000, +0.000]	0/0	3.865	⋆
100	30	30	16.06989	**+0.187**	[+0.109, +0.265]	427/570	3.839	
100	50	50	16.14701	**+0.668**	[+0.582, +0.753]	281/714	3.794	
200	0	uniform	23.38459	**+2.044**	[+1.987, +2.101]	0/994	3.292	
200	5	5	23.03401	**+0.514**	[+0.466, +0.563]	256/744	4.177	
200	10	10	22.95764	**+0.181**	[+0.131, +0.231]	412/588	4.293	
200	20	20	22.91617	+0.000	[+0.000, +0.000]	0/0	4.413	⋆
200	30	30	22.93738	**+0.093**	[+0.038, +0.147]	436/563	4.464	
200	50	50	23.04297	**+0.553**	[+0.495, +0.611]	268/728	4.560	

Reading. $K = 20$ is the optimum at every dimension tested, and the curve is a clean U: too small a list starves the move of good partners, too large a one dilutes it with distant ones that will be rejected. The default needs no change — and notably it does not need to grow with n.

5 Compute cost

What is measured. Two phases: the Clarke & Wright construction, which builds a savings list ($O(n^2)$ exactly, for $n \leq 1500$) and sorts it, and the annealing, which does a fixed number of $O(1)$ steps regardless of n. Instances are solved in parallel by OpenMP, one instance per thread, with no interaction between them.

> **Metric: cumulative CPU time per instance, from the run log — not the `time_ms` column of the CSV.** That column is wall time measured *inside* the 24-thread parallel region, so it absorbs memory contention: at $n = 1000$ it overstates the cost by about 2.5× and is not additive, which makes the annealing look roughly 10× cheaper than it is. Use it for latency distributions (third panel), never for cost accounting.

The annealing cost is isolated as $(t(10S) - t(S))/9$ at equal n: both runs pay the same construction, so it cancels exactly. Construction is then $t(S)$ minus that, cross-checked against an independent `--no-sa` run. A 2× budget delta was tried first and is not enough at $n = 1000$, where the run-to-run spread on the construction alone (±3 %, measured over three repetitions) exceeds the entire annealing cost.

n	construction	annealing	total	annealing share	`--no-sa`
20	0.1171	3.4529	3.5700	96.7 %	0.0040
50	0.1482	3.4978	3.6460	95.9 %	0.0180
100	0.2554	3.5516	3.8070	93.3 %	0.0600
200	0.6731	3.6629	4.3360	84.5 %	0.2060
500	2.7384	3.9216	6.6600	58.9 %	1.2150
1000	18.8184	2.8996	21.7180	13.4 %	19.8340

Table 8: CPU time per instance (ms). Empirical scaling on this grid: construction $\sim n^{1.29}$, annealing $\sim n^{-0.02}$ at a fixed step count — the step count does not depend on n, so that exponent is pure cache behaviour.

threads	wall (s)	speed-up	efficiency
1	2.669	1.00×	100 %
2	1.342	1.99×	99 %
4	0.703	3.80×	95 %
8	0.360	7.41×	93 %
12	0.280	9.53×	79 %
16	0.235	11.36×	71 %
24	0.187	14.27×	59 %

Table 9: Thread scaling at $n = 100$. Instances are independent, so the loss at high thread counts is memory bandwidth and clock throttling, not synchronisation.

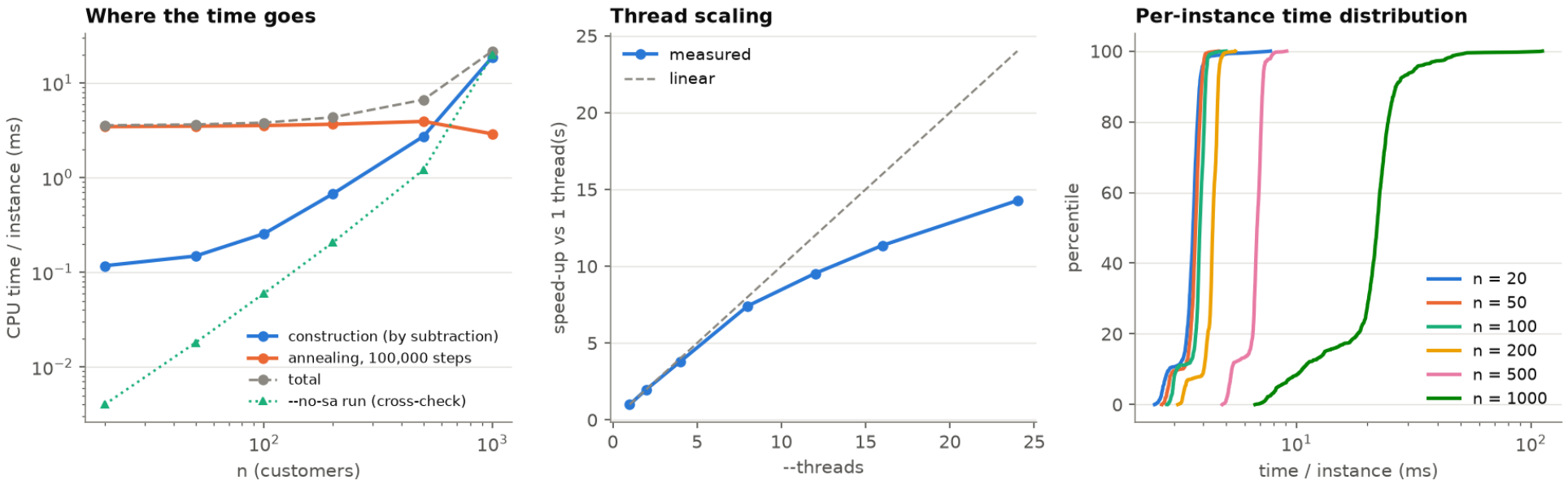

Figure 6: Left: construction and annealing cost against n (log-log). Middle: thread scaling. Right: distribution of per-instance wall time inside the parallel region.

Reading. Up to $n \approx 200$ the annealing is essentially the whole cost and the construction is free; past $n \approx 500$ the $O(n^2)$ savings list takes over and dominates. If large instances ever become the target, the construction is where to look — `--knn` truncates that list, and Section 10 shows the annealing absorbs the loss of quality.

6 Restarts

What it controls. `--restarts R` builds R initial solutions per instance, anneals each one, and keeps the best. Restart 0 is always the deterministic Clarke & Wright; the others are diversified by `--cw-rand`: `perturb` multiplies every saving by $1 + \alpha\, U(-1, 1)$ with $\alpha =$ `--cw-alpha`, which reshuffles the order in which pairs are merged; `param` redraws λ and μ; `both` does both; `off` makes all restarts identical apart from the annealing seed.

> **R multiplies the work by R.** Comparing R restarts against one at the same `--sa-steps` answers nothing useful — of course more compute helps. The study therefore also holds $R \times$ `sa-steps` constant, which asks the question that matters: given a fixed number of annealing steps, is it better to spend them on one long run or several short ones?

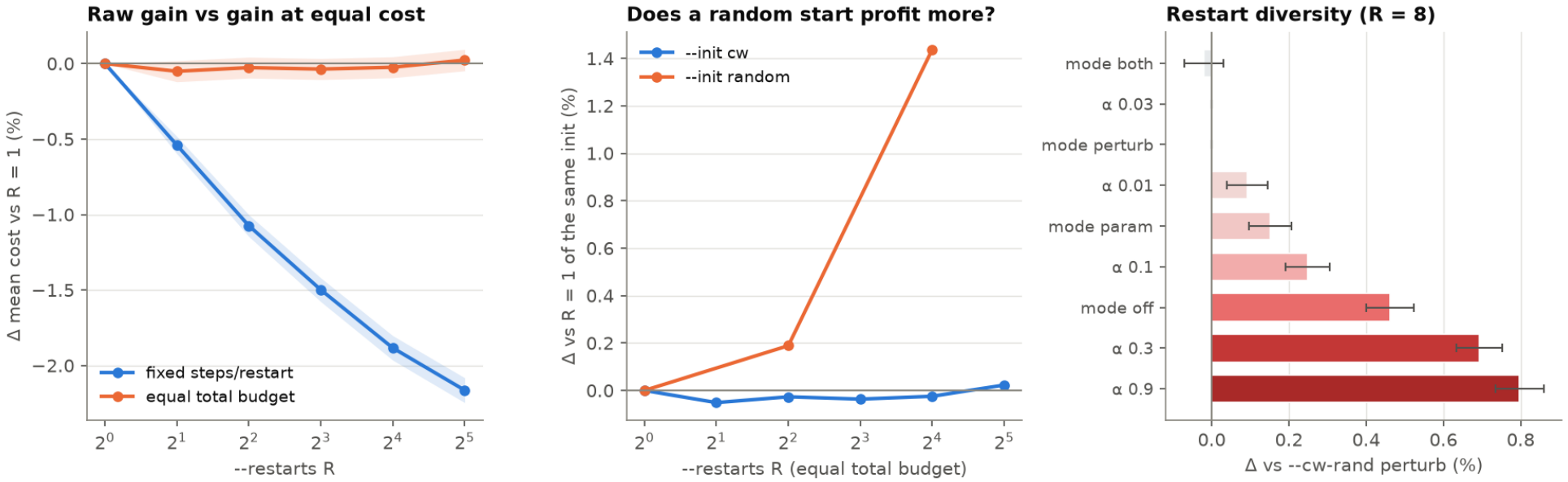

Figure 7: Left: the same restart count judged two ways. Middle: restarts crossed with the initialisation. Right: how the diversity between restarts is generated.

R	steps/restart	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
1	320,000	15.90930	+0.000	[+0.000, +0.000]	0/0	0.56
2	160,000	15.90115	-0.051	[-0.121, +0.018]	508/491	0.56
4	80,000	15.90505	-0.027	[-0.096, +0.042]	507/493	0.58
8	40,000	15.90349	-0.037	[-0.106, +0.033]	491/508	0.60
16	20,000	15.90540	-0.025	[-0.094, +0.045]	490/509	0.65
32	10,000	15.91294	+0.023	[-0.048, +0.094]	465/535	0.73

Table 10: Restarts at *equal total budget* ($R \times$ `sa-steps` $= 320\,000$), paired against $R = 1$.

R	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
1	16.26491	+0.000	[+0.000, +0.000]	0/0	0.03
2	16.17706	**-0.540**	[-0.597, -0.483]	435/0	0.06
4	16.09024	**-1.074**	[-1.145, -1.002]	717/0	0.10
8	16.02116	**-1.499**	[-1.577, -1.420]	855/0	0.19
16	15.95854	**-1.884**	[-1.964, -1.803]	935/0	0.37
32	15.91294	**-2.164**	[-2.246, -2.082]	973/0	0.74

Table 11: Restarts at fixed steps per restart — i.e. at R times the compute. Shown for completeness; the table above is the meaningful one.

variant	mean cost	Δ (%)	95 % CI	win/loss
`mode both`	15.90022	-0.021	[-0.072, +0.031]	463/470
`alpha 0.03`	15.90349	+0.000	[+0.000, +0.000]	0/0
`mode perturb`	15.90349	+0.000	[+0.000, +0.000]	0/0
`alpha 0.01`	15.91803	**+0.091**	[+0.039, +0.144]	432/508
`mode param`	15.92767	**+0.152**	[+0.097, +0.207]	406/537
`alpha 0.1`	15.94303	**+0.249**	[+0.191, +0.306]	342/575
`mode off`	15.97702	**+0.462**	[+0.400, +0.525]	322/628
`alpha 0.3`	16.01370	**+0.693**	[+0.633, +0.753]	165/728
`alpha 0.9`	16.03029	**+0.797**	[+0.735, +0.860]	123/762

Table 12: How restart diversity is produced, at $R = 8$, against the default `perturb` with $\alpha = 0.03$.

Reading. At equal budget, restarts are flat: splitting 320 000 steps into 2, 4, …, 32 independent anneals changes nothing significant. The annealing is not getting trapped in a way that a fresh start would fix, so the whole budget may as well go into one run. The random start is the exception — it degrades sharply as its share of the budget shrinks, which is the same result as Section 2 seen from another angle. On diversity, the default `perturb` at $\alpha = 0.03$ is the best of the options: too much perturbation ($\alpha \geq 0.3$) damages every restart, and `off` is worse still.

7 Optimal Split

What it controls. Split (Vidal, 2016) takes the current solution, concatenates its routes into one giant tour ignoring capacity, then re-cuts that tour into routes optimally by dynamic programming in $O(n)$. Because the current partition is one of the candidates, the result can never be worse. `--split` says where to apply it: `cw` right after the construction, `end` after the annealing, `both`, or `off`. `--split-every N` additionally applies it every N annealing steps — an independent code path (`cw.c:1440` versus `cw.c:1640/1658`), so `off` combined with a period is a meaningful cell. `--split-tour` chooses how the giant tour is built: routes in their current order, `sweep` by polar angle, or `both` keeping the better.

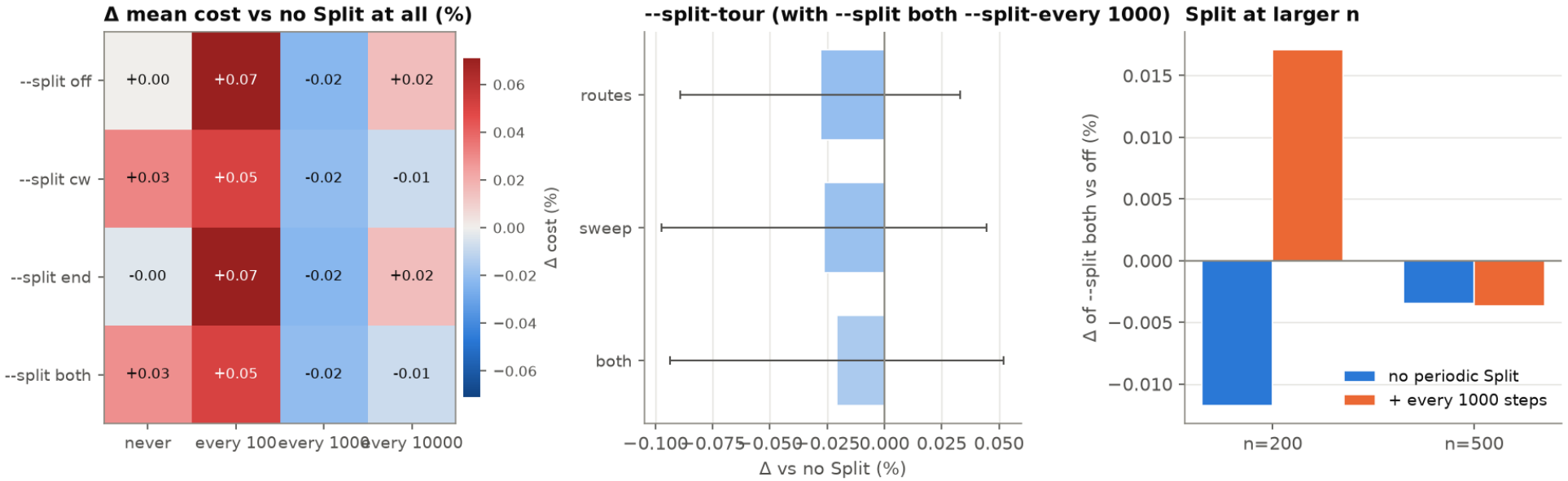

Figure 8: Left: the full mode × period grid. Middle: giant-tour construction. Right: the same comparison at larger n, where there are more routes to repartition.

Table 13: Split mode against periodic Split, paired against no Split at all. 'Split gain' is the total improvement cw attributes to Split internally — note it is large where the paired delta is not, i.e. Split keeps recovering cost the annealing had just spent.

`--split`	`--split-every`	mean cost	Δ (%)	95 % CI	win/loss	Split gain	wall (s)
`off`	never	16.03992	+0.000	[+0.000, +0.000]	0/0	–	0.19
`off`	100	16.05132	+0.071	[-0.002, +0.144]	468/530	0.7104	0.38
`off`	1000	16.03622	-0.023	[-0.094, +0.048]	501/495	0.3170	0.21

--split	--split-every	mean cost	Δ (%)	95 % CI	win/loss	Split gain	wall (s)
off	10000	16.04239	+0.015	[-0.038, +0.069]	467/519	0.0340	0.19
cw	never	16.04496	+0.031	[-0.039, +0.102]	476/518	0.0002	0.18
cw	100	16.04806	+0.051	[-0.020, +0.121]	475/519	0.7314	0.38
cw	1000	16.03661	-0.021	[-0.094, +0.052]	500/498	0.3268	0.21
cw	10000	16.03860	-0.008	[-0.079, +0.062]	496/501	0.0363	0.19
end	never	16.03937	**-0.003**	[-0.005, -0.002]	53/0	0.0006	0.18
end	100	16.05132	+0.071	[-0.002, +0.144]	468/530	0.7104	0.38
end	1000	16.03622	-0.023	[-0.094, +0.048]	501/495	0.3170	0.21
end	10000	16.04238	+0.015	[-0.039, +0.069]	467/519	0.0340	0.19
both	never	16.04451	+0.029	[-0.042, +0.099]	476/518	0.0007	0.18
both	100	16.04806	+0.051	[-0.020, +0.121]	475/519	0.7314	0.38
both	1000	16.03660	-0.021	[-0.094, +0.052]	500/498	0.3268	0.21
both	10000	16.03859	-0.008	[-0.079, +0.062]	497/500	0.0363	0.19

Reading. Split never hurts — `--split end` improves 53 instances and worsens exactly zero, which is the theoretical guarantee showing up in the data — but on this instance family it barely helps either: the best cell is about $-0.02\,\%$, inside the noise. Its internal 'gain' counter is large while the net effect is nil, meaning it mostly undoes damage the annealing did rather than finding new structure. Applying it every 100 steps is actively bad: it costs double the wall time and drags the search back to a repartitioned solution before the annealing can exploit its own moves.

8 Vertex selection

What it controls. Before a move can be built, the annealing must pick the vertex u to disturb (`pick_u`, `cw.c:952`). Uniform choice wastes draws on customers that are already well placed, so the code biases the choice by a *regret* measure. With `--pick` $T \geq 2$ it draws T customers uniformly and keeps the one with the largest regret (a tournament); $T = 1$ is plain uniform; $T = 0$ samples exactly proportional to $\max(\text{regret}, 0) + \varepsilon$ through a Fenwick tree, at $O(\log n)$ per draw and per update.

--pick-crit defines the regret, with $\text{inc}[u] = d(\text{prv}[u], u) + d(u, \text{nxt}[u])$ the cost of the two edges u currently carries:

- `lb` (default) — $\text{inc}[u] - \text{lb}[u]$, the gap to the two shortest edges available to u in its kNN list.
- `rem` — $\text{inc}[u] - d(p, q)$, the removal gain: exactly what relocating u could recover at best.
- `remnorm` — the removal gain divided by $\text{lb}[u]$, normalising by the local density.
- `raw` — $\text{inc}[u]$ itself.

`--pick-eps` sets how peaked the $T = 0$ sampler is.

> The code's own comment (`cw.c:911-916`) notes that `lb` "can stay large for an isolated customer even when nothing can improve it any further", whereas `rem` is zero as soon as u sits well. That suggests the default wastes draws on structurally irreducible vertices. The grid below was run specifically to test that prediction.

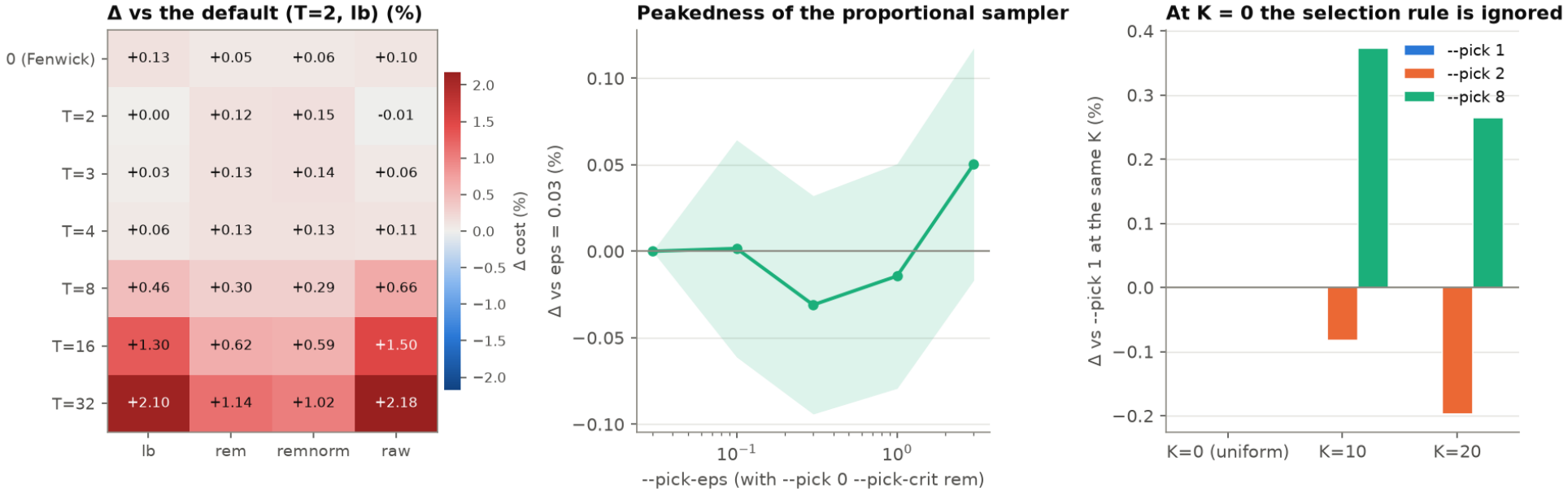

Figure 9: Left: tournament size against regret criterion. Middle: peakedness of the proportional sampler. Right: `--pick` crossed with `--sa-knn`; the flat $K = 0$ group is the silent fallback.

Table 14: Tournament size against regret criterion, paired against the default $T = 2$ with `lb`.

`--pick`	`--pick-crit`	mean cost	Δ (%)	95 % CI	win/loss
1 (uniform)	–	16.07163	**+0.198**	[+0.126, +0.269]	436/563
0 (Fenwick)	`lb`	16.06046	**+0.128**	[+0.058, +0.198]	452/545
0 (Fenwick)	`rem`	16.04842	+0.053	[-0.014, +0.120]	476/520
0 (Fenwick)	`remnorm`	16.04889	+0.056	[-0.012, +0.124]	477/519
0 (Fenwick)	`raw`	16.05559	**+0.098**	[+0.026, +0.170]	461/535
2	`lb`	16.03992	+0.000	[+0.000, +0.000]	0/0
2	`rem`	16.05957	**+0.123**	[+0.054, +0.191]	459/537
2	`remnorm`	16.06360	**+0.148**	[+0.080, +0.216]	430/563
2	`raw`	16.03881	-0.007	[-0.079, +0.065]	490/506
3	`lb`	16.04409	+0.026	[-0.042, +0.094]	487/511
3	`rem`	16.06151	**+0.135**	[+0.067, +0.203]	452/546
3	`remnorm`	16.06199	**+0.138**	[+0.071, +0.205]	453/546
3	`raw`	16.04976	+0.061	[-0.007, +0.130]	480/517

--pick	--pick-crit	mean cost	Δ (%)	95 % CI	win/loss
4	lb	16.04958	+0.060	[-0.011, +0.132]	463/535
4	rem	16.06026	**+0.127**	[+0.059, +0.195]	436/562
4	remnorm	16.06021	**+0.127**	[+0.058, +0.195]	455/540
4	raw	16.05767	**+0.111**	[+0.041, +0.180]	434/562
8	lb	16.11413	**+0.463**	[+0.393, +0.532]	318/678
8	rem	16.08814	**+0.301**	[+0.230, +0.371]	378/618
8	remnorm	16.08644	**+0.290**	[+0.223, +0.357]	386/611
8	raw	16.14616	**+0.662**	[+0.590, +0.735]	270/727
16	lb	16.24908	**+1.304**	[+1.228, +1.380]	118/874
16	rem	16.13911	**+0.618**	[+0.547, +0.690]	301/695
16	remnorm	16.13506	**+0.593**	[+0.522, +0.664]	288/711
16	raw	16.28002	**+1.497**	[+1.422, +1.572]	78/914
32	lb	16.37717	**+2.103**	[+2.023, +2.182]	29/960
32	rem	16.22221	**+1.136**	[+1.060, +1.213]	154/844
32	remnorm	16.20278	**+1.015**	[+0.944, +1.086]	171/829
32	raw	16.38922	**+2.178**	[+2.097, +2.259]	14/974

--sa-knn	--pick	mean cost	Δ (%)	95 % CI	win/loss
0	1	16.40774	+0.000	[+0.000, +0.000]	0/0
10	1	16.09330	+0.000	[+0.000, +0.000]	0/0
20	1	16.07163	+0.000	[+0.000, +0.000]	0/0
0	2	16.40774	+0.000	[+0.000, +0.000]	0/0
10	2	16.08009	**-0.082**	[-0.142, -0.022]	534/449
20	2	16.03992	**-0.197**	[-0.268, -0.126]	563/436
0	8	16.40774	+0.000	[+0.000, +0.000]	0/0
10	8	16.15334	**+0.373**	[+0.309, +0.437]	349/648
20	8	16.11413	**+0.264**	[+0.193, +0.336]	391/606

Table 15: `--pick` crossed with `--sa-knn`, paired against `--pick 1` at the same K. The $K = 0$ rows are exactly zero because `cw.c:1400` forces uniform selection there — while the run header still prints the requested rule.

Reading. The prediction was wrong. At $n = 100$ with 10^5 steps the entire grid — every tournament size, every criterion, and the Fenwick sampler — sits inside $\pm 0.08\,\%$ of the default. The regret machinery is measurable in the code but not in the result: whatever it saves in draw quality it appears to give back in draw cost. It is only at $T \geq 16$ with `lb` or `raw` that the choice starts to hurt, from over-concentrating on a few vertices. The one solid finding here is the silent fallback at $K = 0$: the header reports a rule the code is not using.

9 Annealing schedule

What it controls. The initial temperature T_0 is not given directly: `calibrate_T0` samples worsening moves on the actual instance and solves for the T_0 whose acceptance probability matches `--t-accept` (default 0.001, i.e. 0.1 % of worsening moves accepted at the start). The temperature then decays geometrically to $T_0 \cdot 10^{-D}$ over the whole run, with $D =$ `--t-decades` (default 2). Together they fix both ends of the schedule; `--t0` and `--tend` override the calibration.

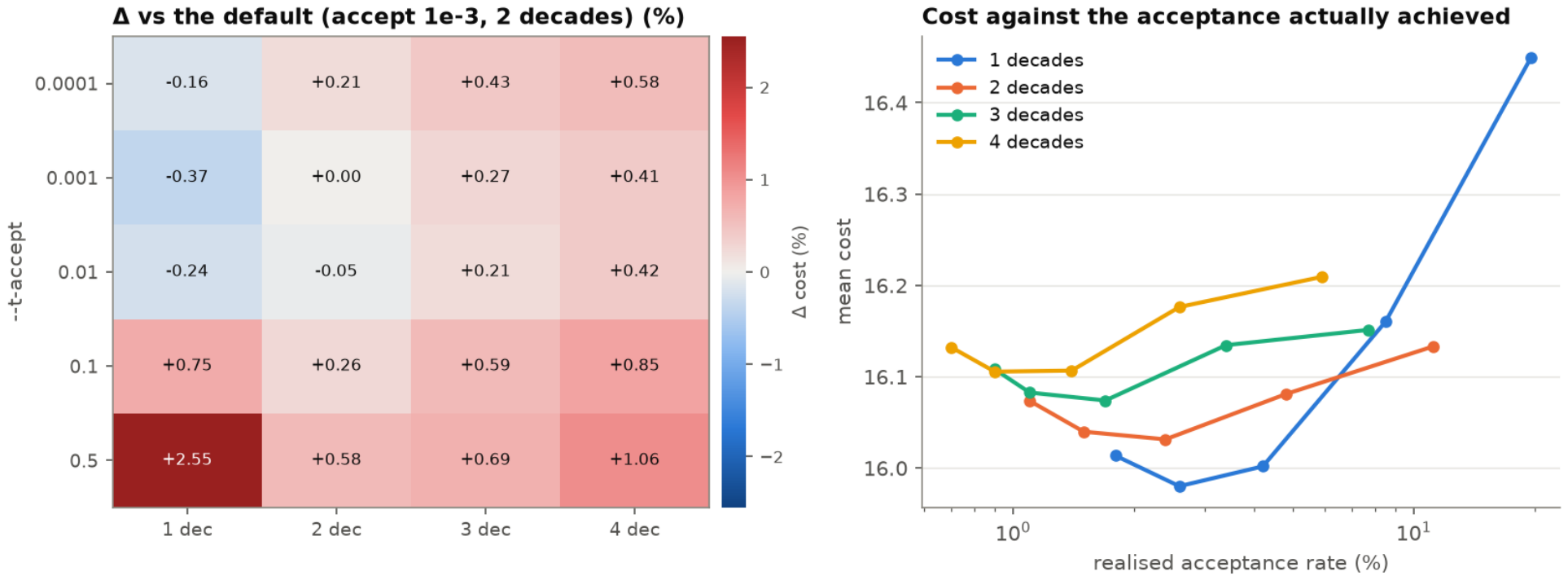

Figure 10: Left: the full `--t-accept` × `--t-decades` grid. Right: the same runs plotted against the acceptance rate they actually realised.

Table 16: Annealing schedule, paired against the default (`t-accept` = 10^{-3}, 2 decades).

`--t-accept`	`--t-decades`	mean cost	accept. (%)	Δ (%)	95 % CI	win/loss
0.0001	1	16.01373	1.80	**-0.163**	[-0.230, -0.097]	547/450
0.0001	2	16.07363	1.10	**+0.210**	[+0.143, +0.278]	427/565
0.0001	3	16.10822	0.90	**+0.426**	[+0.357, +0.495]	348/645

--t-accept	--t-decades	mean cost	accept. (%)	Δ (%)	95 % CI	win/loss
0.0001	4	16.13243	0.70	**+0.577**	[+0.510, +0.643]	294/702
0.001	1	15.98019	2.60	**-0.372**	[-0.445, -0.300]	611/382
0.001	2	16.03992	1.50	+0.000	[+0.000, +0.000]	0/0
0.001	3	16.08271	1.10	**+0.267**	[+0.201, +0.333]	387/604
0.001	4	16.10561	0.90	**+0.410**	[+0.340, +0.479]	339/658
0.01	1	16.00195	4.20	**-0.237**	[-0.319, -0.155]	551/446
0.01	2	16.03146	2.40	-0.053	[-0.135, +0.029]	480/514
0.01	3	16.07404	1.70	**+0.213**	[+0.136, +0.290]	402/592
0.01	4	16.10671	1.40	**+0.416**	[+0.341, +0.491]	348/645
0.1	1	16.16031	8.50	**+0.751**	[+0.663, +0.838]	283/710
0.1	2	16.08115	4.80	**+0.257**	[+0.164, +0.350]	405/589
0.1	3	16.13460	3.40	**+0.590**	[+0.499, +0.681]	318/672
0.1	4	16.17637	2.60	**+0.851**	[+0.759, +0.942]	264/730
0.5	1	16.44932	19.60	**+2.552**	[+2.465, +2.640]	0/984
0.5	2	16.13335	11.20	**+0.583**	[+0.491, +0.674]	301/691
0.5	3	16.15132	7.70	**+0.695**	[+0.600, +0.789]	310/681
0.5	4	16.20941	5.90	**+1.057**	[+0.968, +1.146]	204/784

Reading. This is the largest effect in the whole sweep. Narrowing the temperature range to a single decade is worth about $-0.37\,\%$ — more than every operator, neighbourhood and selection setting combined. The interpretation is that at 10^5 steps the default schedule spends too long at temperatures so low that nothing is accepted: the run is effectively over before the step budget is. Fewer decades keeps the search alive for more of the run. This also implies the right number of decades depends on the budget, so it should be re-checked when `--sa-steps` changes substantially.

10 Construction quality

What it controls. Three separate things about the Clarke & Wright stage. The savings formula $s(i,j) = d_{0i} + d_{0j} - \lambda\, d_{ij} + \mu\,|d_{0i} - d_{0j}|$ has two shape parameters: `--lambda` sets how much weight the direct link between two customers carries against their two depot legs (the classical generalisation of Clarke & Wright, $\lambda > 1$ favouring longer, less radial routes), and `--mu` adds an asymmetry term penalising the merge of two customers at very different distances from the depot. `--knn K` truncates the savings list to each customer's K nearest neighbours instead of all $O(n^2)$ pairs (`--exact` forces the full list; the default is exact for $n \le 1500$). `--2opt` runs an intra-route 2-opt pass after the construction.

The question. Every one of these improves or degrades the *starting* solution. The interesting question is whether that survives the annealing, so each grid is run twice: with `--no-sa` and with the full budget.

Table 17: Savings parameters, construction only, paired against $\lambda = 1$, $\mu = 0$.

λ	μ	**mean cost**	Δ (%)	**95 % CI**	**win/loss**
0.6	0	17.04835	**+3.639**	[+3.496, +3.782]	42/958
0.6	0.2	17.07377	**+3.794**	[+3.657, +3.931]	34/966
0.6	0.5	17.08990	**+3.892**	[+3.753, +4.031]	39/961
0.6	1	17.13273	**+4.152**	[+3.996, +4.308]	52/948
0.8	0	16.65490	**+1.247**	[+1.140, +1.354]	207/793
0.8	0.2	16.72637	**+1.682**	[+1.571, +1.792]	142/858
0.8	0.5	16.81977	**+2.250**	[+2.126, +2.374]	135/865
0.8	1	17.00364	**+3.367**	[+3.218, +3.517]	88/912
1	0	16.44971	+0.000	[+0.000, +0.000]	0/0
1	0.2	16.50233	**+0.320**	[+0.228, +0.412]	395/605
1	0.5	16.61998	**+1.035**	[+0.919, +1.151]	288/712
1	1	16.87703	**+2.598**	[+2.461, +2.734]	128/872
1.2	0	16.44126	-0.051	[-0.148, +0.045]	514/485
1.2	0.2	16.37879	**-0.431**	[-0.525, -0.337]	574/402
1.2	0.5	16.47483	**+0.153**	[+0.043, +0.262]	459/541
1.2	1	16.76165	**+1.896**	[+1.761, +2.031]	206/794
1.4	0	16.50587	**+0.341**	[+0.227, +0.456]	425/575
1.4	0.2	16.38595	**-0.388**	[-0.495, -0.281]	598/402
1.4	0.5	16.38419	**-0.398**	[-0.509, -0.288]	584/416
1.4	1	16.66358	**+1.300**	[+1.169, +1.431]	266/734

Table 18: Savings parameters, after annealing, paired against $\lambda = 1$, $\mu = 0$.

λ	μ	**mean cost**	Δ (%)	**95 % CI**	**win/loss**
0.6	0	16.15556	**+0.721**	[+0.608, +0.834]	350/650
0.6	0.2	16.13088	**+0.567**	[+0.460, +0.674]	369/631
0.6	0.5	16.07704	**+0.231**	[+0.135, +0.328]	423/577
0.6	1	16.03118	-0.055	[-0.152, +0.043]	515/485
0.8	0	16.08606	**+0.288**	[+0.191, +0.385]	442/558
0.8	0.2	16.06927	**+0.183**	[+0.090, +0.276]	454/545
0.8	0.5	16.05291	+0.081	[-0.015, +0.177]	488/512
0.8	1	16.02041	**-0.122**	[-0.219, -0.024]	531/469
1	0	16.03992	+0.000	[+0.000, +0.000]	0/0
1	0.2	16.04161	+0.011	[-0.075, +0.096]	489/511
1	0.5	16.02794	-0.075	[-0.167, +0.018]	517/483
1	1	16.01006	**-0.186**	[-0.281, -0.091]	538/462

λ	μ	mean cost	Δ (%)	95 % CI	win/loss
1.2	0	16.02419	-0.098	[-0.187, -0.009]	513/485
1.2	0.2	16.01470	**-0.157**	[-0.241, -0.073]	526/450
1.2	0.5	16.01161	**-0.176**	[-0.266, -0.086]	534/466
1.2	1	16.00484	**-0.219**	[-0.314, -0.124]	552/448
1.4	0	16.00424	**-0.222**	[-0.312, -0.133]	555/445
1.4	0.2	15.99402	**-0.286**	[-0.375, -0.197]	565/435
1.4	0.5	16.00320	**-0.229**	[-0.320, -0.138]	556/444
1.4	1	16.00195	**-0.237**	[-0.331, -0.143]	553/447

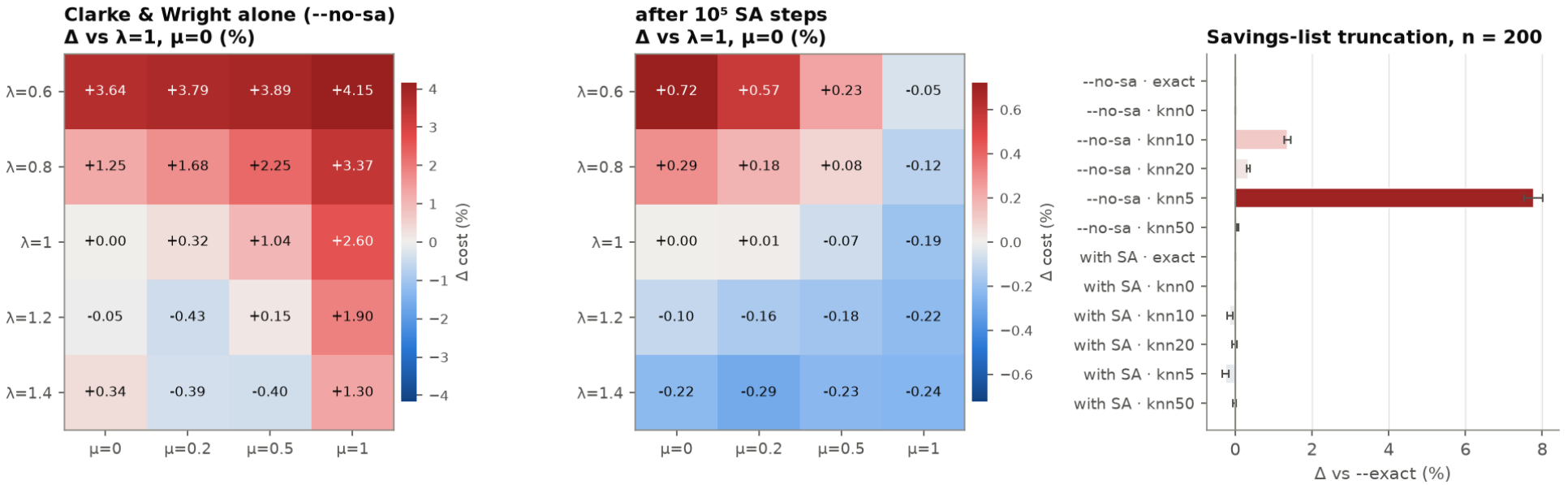

Figure 11: Left and middle: the same $\lambda \times \mu$ grid before and after annealing — note the colour scales differ by an order of magnitude. Right: truncating the savings list, with and without annealing.

stage	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
`--no-sa`	`exact`	23.38509	+0.000	[+0.000, +0.000]	0/0	0.01
`--no-sa`	`knn0`	23.38509	+0.000	[+0.000, +0.000]	0/0	0.01
`--no-sa`	`knn10`	23.70418	**+1.365**	[+1.272, +1.457]	135/865	0.01
`--no-sa`	`knn20`	23.46603	**+0.346**	[+0.304, +0.388]	177/768	0.02
`--no-sa`	`knn5`	25.20278	**+7.773**	[+7.526, +8.019]	3/997	0.01
`--no-sa`	`knn50`	23.40135	**+0.070**	[+0.054, +0.085]	76/429	0.04
`with SA`	`exact`	22.91617	+0.000	[+0.000, +0.000]	0/0	0.21
`with SA`	`knn0`	22.91617	+0.000	[+0.000, +0.000]	0/0	0.21
`with SA`	`knn10`	22.88403	**-0.140**	[-0.211, -0.070]	539/461	0.21
`with SA`	`knn20`	22.91147	-0.020	[-0.077, +0.036]	476/469	0.20
`with SA`	`knn5`	22.85942	**-0.248**	[-0.332, -0.164]	574/426	0.22
`with SA`	`knn50`	22.91173	-0.019	[-0.056, +0.018]	259/246	0.24

Table 19: Savings-list truncation at $n = 200$, paired against `--exact`. `knn0` is the automatic setting, which is exact at this size.

stage	without `--2opt`	with `--2opt`	Δ (%)	95 % CI	win/loss
construction only	16.44971	16.37485	**-0.455**	[-0.476, -0.434]	962/0
after annealing	16.03992	16.04370	+0.024	[-0.044, +0.092]	462/499

Table 20: Intra-route 2-opt after the construction.

Reading. Two opposite lessons. The savings *shape* matters and survives: $\lambda = 1.2$–1.4 with $\mu \approx 0.2$–0.5 is worth about $-0.29\,\%$ after annealing, second only to the schedule. The savings *list*, by contrast, does not survive at all: truncating it to the 5 nearest neighbours costs $+7.7\,\%$ on the raw construction and essentially nothing once the annealing has run. That is a useful lever for large n, where Section 5 showed the $O(n^2)$ list is the dominant cost — the annealing repairs what the truncation breaks. `--2opt` is likewise absorbed: it improves the construction and changes nothing afterwards.

11 Do the knobs combine?

What is tested. Tuning one knob at a time says nothing about tuning several at once. These runs take three plausible changes, apply them individually and together at equal SA budget, and compare against the stock defaults. `all_r8` additionally splits the same budget over 8 restarts.

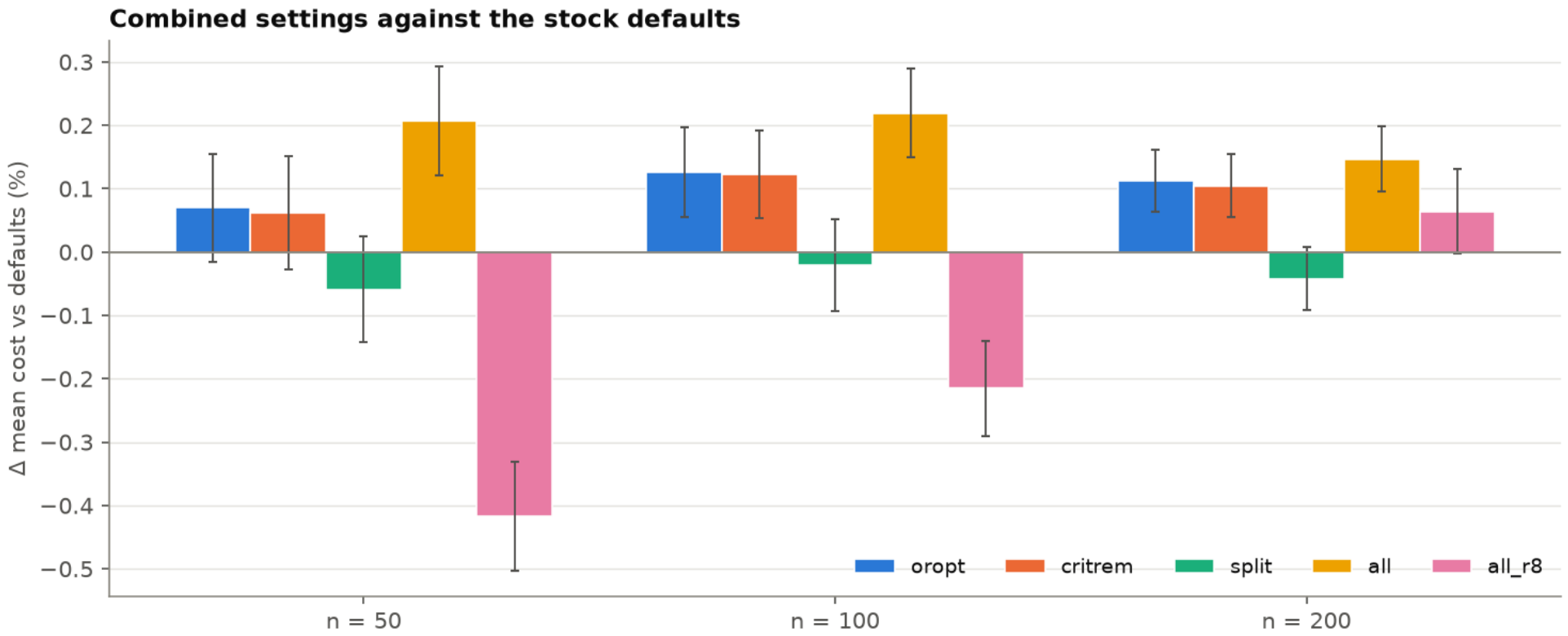

Figure 12: Candidate combinations against the stock defaults at equal SA budget, with 95 % CIs.

Table 21: Combinations against the stock defaults.

n	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
50	or-opt enabled	10.53862	+0.070	[-0.015, +0.156]	440/493	0.18
100	or-opt enabled	16.06019	**+0.126**	[+0.055, +0.198]	456/541	0.19
200	or-opt enabled	22.94209	**+0.113**	[+0.064, +0.162]	440/560	0.22
50	pick-crit rem	10.53778	+0.062	[-0.027, +0.151]	444/505	0.17
100	pick-crit rem	16.05957	**+0.123**	[+0.054, +0.191]	459/537	0.18
200	pick-crit rem	22.94017	**+0.105**	[+0.055, +0.154]	467/532	0.21
50	Split both + every 1000	10.52504	-0.059	[-0.142, +0.024]	484/442	0.19
100	Split both + every 1000	16.03660	-0.021	[-0.094, +0.052]	500/498	0.21

n	variant	mean cost	Δ (%)	95 % CI	win/loss	wall (s)
200	Split both + every 1000	22.90653	-0.042	[-0.091, +0.007]	512/487	0.26
50	the three combined	10.55309	**+0.208**	[+0.121, +0.294]	430/521	0.20
100	the three combined	16.07514	**+0.220**	[+0.150, +0.289]	394/601	0.21
200	the three combined	22.94986	**+0.147**	[+0.096, +0.198]	425/574	0.26
50	the three, over 8 restarts	10.48731	**-0.417**	[-0.503, -0.331]	594/373	0.22
100	the three, over 8 restarts	16.00540	**-0.215**	[-0.290, -0.140]	554/446	0.26
200	the three, over 8 restarts	22.93080	+0.064	[-0.003, +0.131]	420/580	0.37

Reading. The combination is worse than its parts: `all` loses to the defaults at every n, even though one of its three ingredients helps on its own. Gains found one knob at a time do not add up, which is exactly why the recommendation in Section 12 is confirmed by re-running it rather than assembled on paper.

12 The configuration to use

A knob enters the command below only if its best setting beat *its own* default significantly — both the 95 % CI excluding zero and the sign test rejecting at 5 %. Everything else stays at the default, deliberately: a knob whose interval straddles zero has not earned a change.

knob	setting	Δ (%)	95 % CI	win/loss	flags
schedule	`a0.001_d1`	**-0.372**	[-0.445, -0.300]	611/382	`--t-decades 1`
savings shape	`sa_l1.4_m0.2`	**-0.286**	[-0.375, -0.197]	565/435	`--lambda 1.4 --mu 0.2`

Table 22: Knobs that earned their place, with the improvement each showed on its own.

knob	best setting tried	Δ (%)	95 % CI	win/loss
operators	`sub_1110`	+0.000	[+0.000, +0.000]	0/0
or-opt length	`ormax_7`	-0.007	[-0.075, +0.062]	503/496
neighbourhood	`n100_K20`	+0.000	[+0.000, +0.000]	0/0
vertex choice	`T2_raw`	-0.007	[-0.079, +0.065]	490/506
Split	`mend_e1000`	-0.023	[-0.094, +0.048]	501/495
restarts (equal budget)	`iso_R2`	-0.051	[-0.121, +0.018]	508/491
initialisation	`n100_cw_s100000`	+0.000	[+0.000, +0.000]	0/0

Table 23: Knobs left at their default: no significant improvement.

12.1 The command

Written out in full, defaults included, so the run does not depend on what `cw`'s built-in defaults happen to be:

```
./cw --bundle data/cvrp_100.cvrpb \
     --init cw \
     --knn 0 --lambda 1.4 --mu 0.2 \
     --sa-steps 100000 --ops 1,1,1,0 --or-max 3 \
     --t-accept 0.001 --t-decades 1 \
     --sa-knn 20 --pick 2 --pick-crit lb --pick-eps 0.3 \
     --restarts 1 --cw-rand perturb --cw-alpha 0.03 \
     --split off --split-every 0 --split-tour both
```

On generated instances instead of a bundle, swap the first line for `--random -n 100 -m 1000 --seed 42`:

```
./cw --random -n 100 -m 1000 --seed 42 \
     --init cw \
     --knn 0 --lambda 1.4 --mu 0.2 \
     --sa-steps 100000 --ops 1,1,1,0 --or-max 3 \
     --t-accept 0.001 --t-decades 1 \
     --sa-knn 20 --pick 2 --pick-crit lb --pick-eps 0.3 \
     --restarts 1 --cw-rand perturb --cw-alpha 0.03 \
     --split off --split-every 0 --split-tour both
```

Only `--lambda`, `--mu` and `--t-decades` differ from the stock defaults (`--lambda` was `1`, `--mu` was `0` and `--t-decades` was `2`). Every other value above is what `cw` would have used anyway, written out so the run is reproducible against a future version whose defaults have moved.

12.2 What the three changed options do

`--t-decades 1` (default 2) — the width of the temperature schedule. The annealing does not take T_0 as an argument: `calibrate_T0` samples worsening moves on the instance itself and solves for the T_0 at which a fraction `--t-accept` of them would be accepted (0.1 % by default). From there the temperature decays geometrically to $T_{\text{end}} = T_0 \cdot 10^{-D}$, one factor $\alpha = 10^{-D/(\text{steps}-1)}$ per step, with $D =$ `--t-decades`. So D is *how many orders of magnitude the temperature falls over the run*: $D = 2$ ends a hundred times colder than it started, $D = 1$ only ten times colder.

Lowering D keeps the search warm for longer. The realised acceptance rate over the whole run goes 0.9 % at $D = 4$, 1.1 % at $D = 3$, 1.5 % at the default $D = 2$, 2.6 % at $D = 1$ — and the cost falls monotonically along that sequence. The reading is that the default schedule freezes too early: it reaches temperatures at which essentially nothing is accepted well before the step budget runs out, and those steps are wasted. Widening the schedule further ($D = 3$, $D = 4$) makes it strictly worse, by +0.27 % and +0.41 %.

`--lambda 1.4` **and** `--mu 0.2` (defaults 1 and 0) — the shape of the Clarke & Wright savings criterion,

$$s(i,j) \;=\; d_{0i} + d_{0j} \;-\; \lambda\, d_{ij} \;+\; \mu\, |d_{0i} - d_{0j}|.$$

At $\lambda = 1$, $\mu = 0$ this is the original 1964 criterion: the distance saved by serving i and j on one route rather than two separate out-and-back trips. The construction merges route endpoints in decreasing s while capacity allows, so s decides nothing but the *order* in which pairs are considered — which is enough to determine the whole solution.

λ reweights the direct link d_{ij} against the two depot legs. Raising it above 1 makes the criterion more sceptical of long links, so only genuinely close pairs are merged early; the routes that come out are more compact and less radial. μ adds a bonus for merging two customers that sit at *different* distances from the depot, which favours routes that run outward and sweep back rather than joining two customers on the same ring.

The two interact, and neither is any good alone: at $\lambda = 1.4$, $\mu = 0$ the construction is $+0.34\,\%$ *worse* than the default, and $\mu = 1$ at the same λ is $+1.30\,\%$ worse. It is the combination $\lambda \approx 1.4$ with a small positive μ that wins, and that is a fact about this instance family — uniform points on a square, where the depot is not central by construction — not a universal constant. On clustered instances the optimum will move.

> **Both of these are conditional on the budget.** `--t-decades` was tuned at `--sa-steps` $= 100,000$ and controls how fast the temperature reaches the point where nothing is accepted; that trade-off is different at 10^4 or 10^6 steps. Re-run the `temp` study if you change the budget substantially. The savings parameters are the more portable of the two, but they were still tuned on one instance family.

12.3 Confirmation

The combination is re-run from scratch, because Section 11 showed that knobs tuned separately do not add up. It is checked on the sweep's own instances and on a seed the sweep never saw — the difference between the two is the selection bias made visible.

instances	n	default	recommended	Δ (%)	95 % CI	win/loss
sweep seed (in-sample)	50	10.53122	10.48610	**-0.429**	[-0.527, -0.330]	566/392
sweep seed (in-sample)	100	16.03992	15.95096	**-0.555**	[-0.642, -0.468]	628/372
sweep seed (in-sample)	200	22.91617	22.72582	**-0.831**	[-0.913, -0.748]	727/273
fresh seed	50	10.55668	10.51511	**-0.394**	[-0.487, -0.300]	573/391
fresh seed	100	15.99497	15.91590	**-0.494**	[-0.581, -0.408]	624/376
fresh seed	200	22.93715	22.74684	**-0.830**	[-0.910, -0.749]	741/259

Table 24: The recommended configuration against the stock defaults, at equal SA budget. The fresh-seed rows are the honest estimate.

13 Caveats

- **One instance family.** Uniform coordinates and uniform demands. Nothing here is evidence about clustered or real-world instances; run `tools/fetch_neuopt.py` and point the sweep at `--bundle` for that.
- **One budget.** Most grids are at 10^5 annealing steps. The schedule result in particular is budget-dependent by construction.
- **Selection bias.** The overview picks each knob's best setting *because* it scored lowest. The fresh-seed rows in Section 12 are the only bias-free numbers in this document.

- `--sa-knn` **is clamped** to $n-1$ (`cw.c:1399`), so at $n=20$ the settings $K \geq 20$ are the same run; and at $K=0$ the vertex-selection rule is silently forced to uniform (`cw.c:1400`) while the header still prints the requested `--pick`. That is a reporting bug worth fixing in `cw`.